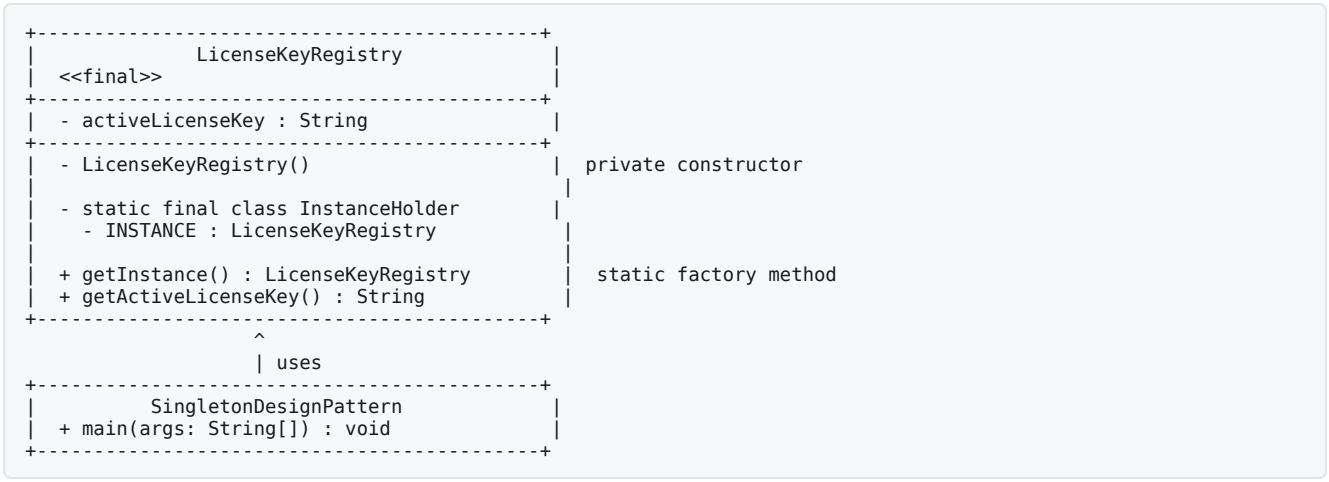
Singleton Design Pattern

Intent

Ensure a class has **exactly one instance** for the entire lifetime of the JVM, and provide a single, well-known, global access point to reach it. No other code — anywhere in the application — is able to construct a second one, because construction itself is placed under the class's own control.

This is the **canonical GoF Singleton**: a private constructor closes off direct instantiation, and a `public static` accessor is the only door in. The example models a licensing subsystem — an application-wide `LicenseKeyRegistry` — the kind of object every real system has exactly one of: an app process is activated under exactly one license key at a time, and every component that gates a feature on licensing must see the same, mutually consistent answer.

UML class diagram



The players

```
com.design.patterns.singleton.LicenseKeyRegistry        the Singleton — one instance, globally reachable
                              .InstanceHolder             private nested class; owns the one INSTANCE field
com.design.patterns.SingletonDesignPattern                the client / driver — calls getInstance(), proves
identity
```

The code, line by line

`LicenseKeyRegistry.java`

```
package com.design.patterns.singleton;

public final class LicenseKeyRegistry {

    private final String activeLicenseKey;

    private LicenseKeyRegistry() {
        this.activeLicenseKey = "ENT-7734-9A2F-PROD";
    }

    private static final class InstanceHolder {
        private static final LicenseKeyRegistry INSTANCE = new LicenseKeyRegistry();
    }

    public static LicenseKeyRegistry getInstance() {
        return InstanceHolder.INSTANCE;
    }

    public String getActiveLicenseKey() {
        return activeLicenseKey;
```

```
    }
}
```

Line by line:

- `package com.design.patterns.singleton;` — Declares that this class lives in its own dedicated sub-package, separate from the driver class. This keeps the Singleton's implementation independently importable and mirrors how every other pattern in this reactor organizes its "core" classes under a pattern-named package.
- `public final class LicenseKeyRegistry {` — A `public` top-level class so any package can reference it, and `final` so it can never be subclassed. Subclassing matters here because a subclass could otherwise be instantiated independently of the parent's controlled access point if any constructor path were reachable from it — `final` removes that escape hatch entirely.
- `private final String activeLicenseKey;` — A single instance field, `final` because once the one instance is built its state should not silently drift; every caller that reads `activeLicenseKey` should see the same, immutable value for the life of the JVM. This also demonstrates that the Singleton is not a bare marker class — it carries real state, the way a real licensing registry would.
- `private LicenseKeyRegistry() {` — The constructor is `private`. This is the entire mechanism of the pattern: no code outside this class — not even a subclass, since the class is `final` and has none — can write `new LicenseKeyRegistry()`. The class alone decides when, and how many times, it is built.
- `this.activeLicenseKey = "ENT-7734-9A2F-PROD";` — The one and only place the field is ever set. In a real system this is where you'd read the activation key from a license file, an environment variable, or a licensing server's activation response — once, at construction time.
- `}` — Closes the constructor.
- `private static final class InstanceHolder {` — A `private static` nested class whose only purpose is to hold the single instance. `private` hides it from every class outside `LicenseKeyRegistry`, so nothing external can ever reference `InstanceHolder` directly. `static` means it carries no implicit reference to an enclosing `LicenseKeyRegistry` object — it doesn't need one, because at the point this class is loaded no instance exists yet.
- `private static final LicenseKeyRegistry INSTANCE = new LicenseKeyRegistry();` — A `private static final` field, initialized with the *one and only* call to the private constructor anywhere in this file. Being `static`, it belongs to `InstanceHolder`, not to any object — there is exactly one slot for it, ever. Being `final`, the reference can never be reassigned after this line runs.
- `}` — Closes `InstanceHolder`.
- `public static LicenseKeyRegistry getInstance() {` — The single public entry point. `static` because it must be callable before any instance exists — that's the whole point of a factory method that hands out the one instance. `public` because every caller, in any package, needs to be able to reach the Singleton through this door.
- `return InstanceHolder.INSTANCE;` — Reads the field. See **Why the holder idiom is thread-safe with no explicit locking** below for exactly what happens the first time this line runs versus every time after.
- `}` — Closes `getInstance()`.
- `public String getActiveLicenseKey() {` / `return activeLicenseKey;` / `}` — An ordinary getter, present so the returned object is demonstrably useful and not just an empty shell. It proves the Singleton carries and serves real state to whoever holds a reference to it.
- `}` — Closes `LicenseKeyRegistry`.

`SingletonDesignPattern.java`

```
package com.design.patterns;

import com.design.patterns.singleton.LicenseKeyRegistry;

public class SingletonDesignPattern {

    public static void main(String[] args) {
        LicenseKeyRegistry first = LicenseKeyRegistry.getInstance();
        LicenseKeyRegistry second = LicenseKeyRegistry.getInstance();

        System.out.println("Both references point to the same instance: " + (first == second));
        System.out.println("Active license key: " + first.getActiveLicenseKey());
    }
}
```

Line by line:

- `package com.design.patterns;` — The driver sits in the shared top-level package for this reactor, one level above the pattern-specific `singleton` sub-package that holds the actual implementation. This mirrors how every module in this repository separates its "client/demo" class from its "pattern implementation" classes.
- `import com.design.patterns.singleton.LicenseKeyRegistry;` — Brings the Singleton class into scope by its simple name.
- `public class SingletonDesignPattern {` — `public` is required so the JVM launcher can resolve `main` by this class's fully-qualified name from the command line.
- `public static void main(String[] args) {` — The standard Java entry point: `static` so it runs without needing an instance of `SingletonDesignPattern` itself (there's no reason this driver class should ever be instantiated).
- `LicenseKeyRegistry first = LicenseKeyRegistry.getInstance();` — The first call ever made to `getInstance()`. This is the call that triggers construction, as traced in Execution flow below.
- `LicenseKeyRegistry second = LicenseKeyRegistry.getInstance();` — A second call. No new object is built — `second` receives the exact same reference `first` did.
- `System.out.println("Both references point to the same instance: " + (first == second));` — `==` on objects compares **references** (identity), not field contents. This line exists specifically to prove, empirically, that the pattern's core guarantee holds: `first` and `second` are literally the same object in memory, not two equal-looking copies.
- `System.out.println("Active license key: " + first.getActiveLicenseKey());` — Exercises the Singleton through the reference obtained first, proving the shared instance actually does useful work and carries real state.
- `}` `}` — Close `main` and the class.

Why these design decisions

Why `final` on the class?

If a subclass existed, and any constructor of the hierarchy were reachable from outside, external code could construct a second, distinct object that is-a `LicenseKeyRegistry` but is not *the* `LicenseKeyRegistry`. `final` removes the possibility of a subclass entirely, so the "exactly one instance" guarantee cannot be bypassed by extension.

Why the Initialization-on-demand holder idiom instead of eager initialization or double-checked locking?

- **Eager initialization** — declaring `private static final LicenseKeyRegistry INSTANCE = new LicenseKeyRegistry();` directly on `LicenseKeyRegistry` itself — works and is thread-safe, but is not *lazy*: the instance is built the moment the JVM loads the class, even if `getInstance()` is never called. That's wasteful if construction is expensive (a real license registry might validate the key against a signature or call out to an activation server) and the class happens to be loaded but unused on some code path.
- **Double-checked locking** (a `volatile` field plus a `synchronized` block with two null-checks) is correct in Java 5+, but it requires the reader to trust the Java Memory Model to believe it: why `volatile` is mandatory to prevent unsafe publication of a partially-constructed object, why the lock alone isn't enough, why the null-check has to happen twice. It works, but it is easy to get subtly wrong when hand-written, and it adds visible synchronization machinery to code that, with the holder idiom, needs none.
- **The holder idiom (used here)** gets laziness, thread safety, and zero lock overhead simultaneously, by leaning on a guarantee the JVM already has to provide for reasons unrelated to this pattern.

Why the holder idiom is thread-safe with no explicit locking

The mechanism rests entirely on one guarantee from the Java Language Specification: **a class is initialized at most once, at the point of its first active use, and the JVM's class-initialization process is itself synchronized.**

Concretely:

1. `InstanceHolder` is a nested class that is *not* referenced anywhere except inside `getInstance()`. Because the JVM only loads and initializes a class when it is first actively used, `InstanceHolder` — and therefore its `INSTANCE` field, and therefore the `new LicenseKeyRegistry()` call — does not run until the first thread calls `getInstance()`. That is what makes it **lazy**.
2. When a class is initialized, the JVM acquires an internal per-class initialization lock before running its static initializers, and does not release it until initialization completes. If a second thread calls `getInstance()` while the first thread's class-initialization is still in progress, the JVM blocks that second thread at the class-loading level — it cannot read `InstanceHolder.INSTANCE` until the first thread's initialization has fully finished and published its result. This is what makes it **thread-safe**: two threads racing into `getInstance()` for the first time cannot both win and construct two different objects, because the JVM itself serializes the one construction that matters, with no `synchronized` keyword anywhere in *this* code.
3. Once `InstanceHolder` has been initialized, the JVM records that fact permanently. Every later call to `getInstance()` — from any thread — sees `InstanceHolder` as already-initialized and reads `INSTANCE` directly,

with no lock acquisition, no `volatile` read barrier, and no branching. That is what makes the **steady-state cost zero**: after the very first call, `getInstance()` is exactly as fast as reading a plain static field.

In short: double-checked locking manually reimplements a safe-publication guarantee that the class loader already gives you for free. The holder idiom simply arranges for the object's construction to *be* a class initialization, and lets the JVM's own (already-correct, already-optimized) locking do the work instead of hand-rolled `synchronized` / `volatile` code.

Why a nested `private static` class instead of putting `INSTANCE` directly on `LicenseKeyRegistry`?

If `INSTANCE` were a static field directly on `LicenseKeyRegistry`, it would be initialized the moment `LicenseKeyRegistry` itself is loaded — which happens as soon as *any* static member of `LicenseKeyRegistry` is resolved, including, awkwardly, class-loading triggered by simply referencing `LicenseKeyRegistry.getInstance` before it's even called. Moving `INSTANCE` into a separate nested class means `LicenseKeyRegistry` can be loaded (its constructor and `getActiveLicenseKey()` method resolved, its bytecode verified) **without** forcing construction — only touching `InstanceHolder` (which only happens inside `getInstance()`) triggers it. That separation of "the class exists" from "the instance is built" is precisely what buys the laziness.

Why `public static` for `getInstance()`?

`static` is mandatory: there is, by definition, no instance to invoke a method on before the caller has retrieved one. `public` is mandatory for the opposite reason: this method **is** the pattern's entire public contract — the one supported way for any caller, in any package, to reach the shared object.

Trade-offs worth knowing

- **Reflection** can still call the private constructor via `setAccessible(true)`, silently creating a second instance. A defensive constructor can throw if `InstanceHolder.INSTANCE` already has a value, but this example keeps the classic, textbook-clean form.
- **Serialization** would construct a fresh object on deserialization via `ObjectInputStream`, bypassing `getInstance()` entirely; a real implementation would add `readResolve()` returning `getInstance()`.
- **Testability** — a global singleton is global state, which can make substituting a fake implementation in unit tests awkward. Production code typically injects the singleton behind an interface so tests can supply a different implementation without touching the static accessor.

Execution flow (as run from `main`)

```
SingletonDesignPattern.main
        │
        ├── LicenseKeyRegistry.getInstance()        first call, from "first"
        │       │
        │       └── InstanceHolder.INSTANCE          first-ever reference to InstanceHolder
        │               │
        │               ├── JVM acquires the class-init lock for InstanceHolder
        │               ├── static field initializer runs: new LicenseKeyRegistry()
        │               │       └── activeLicenseKey = "ENT-7734-9A2F-PROD"
        │               ├── INSTANCE now holds the one and only object
        │               └── JVM releases the class-init lock; InstanceHolder is now "initialized"
        │       └── returns InstanceHolder.INSTANCE  →  first
        │
        ├── LicenseKeyRegistry.getInstance()        second call, from "second"
        │       └── InstanceHolder already initialized → direct field read, no lock
        │       └── returns InstanceHolder.INSTANCE  →  second (== first)
        │
        ├── first == second                         true — same object in memory
        │       └── prints "Both references point to the same instance: true"
        │
        └── first.getActiveLicenseKey()              "ENT-7734-9A2F-PROD"
                └── prints "Active license key: ENT-7734-9A2F-PROD"
```

Expected output

```
Both references point to the same instance: true
Active license key: ENT-7734-9A2F-PROD
```

(Captured from a real run — see **How to run** below.)

How to run

```
# From the module root directory
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 mvn -o clean compile

JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 java -cp target/classes com.design.patterns.SingletonDesignPattern
```

The `-o` (offline) flag skips network calls when the parent reactor's dependencies are already in the local Maven cache. This module carries no Spring Boot dependency and no framework wiring — `getInstance()` is called directly, which is the point: the Singleton pattern is a plain-Java, class-loader-level guarantee, not a container-managed one.

Singleton vs. a Spring `@Component`

A Spring bean declared without `@Scope("prototype")` is *also* effectively a singleton — but for a different reason and under a different authority: the `ApplicationContext` decides to hand out the same bean instance, and that guarantee only holds *within that one context*. Two different Spring contexts in the same JVM will each build their own "singleton" bean. The GoF Singleton demonstrated here makes the guarantee at the **class level**, enforced by the private constructor and the class loader, and it holds for the entire JVM regardless of any framework being present at all.

Other real-world problems this pattern can solve

- **License activation registry** — Validate an activation key once against a signature or licensing server at process start, then expose the single validated key to every feature gate, so no component re-validates or re-parses it redundantly.
- **Hardware device handle** — A serial port, GPU context, or printer spooler often can only be opened by one owner at a time; a Singleton models the one handle every caller must share.
- **Metrics registry** — A single in-process counter/gauge registry so every component increments the same set of metrics rather than each keeping an isolated, unreported copy.
- **Feature-toggle snapshot** — A single point-in-time snapshot of which features are enabled, read once at startup so every request in flight sees a consistent toggle state instead of a mid-request change.
- **Sequence/ID generator** — One shared counter guarantees no two callers ever hand out the same identifier, which would be impossible to guarantee with independently-constructed generators.